

Obsah

1	Databaze ručně.....	2
2	Nastavení databáze:.....	3
3	Vytvoření entity.....	3
4	Vytvoříme Controller.....	4
5	Managery pro práci s daty.....	4
5.1	EntityManager:.....	4
6	Zápis dat do databáze.....	4
7	Zapis do databáze.....	4
8	Čtení z databáze.....	5
8.1	Použití EntityManageru.....	5
8.1.1	Výpis pomocí Responze.....	5
8.1.2	Použití findall a výpis pomocí responze.....	5
8.1.3	Výpis pomocí twigu.....	6
8.1.4	Poznámka pro find.....	7
8.1.5	Použití find().....	7
8.2	Použití ManagerRegistry.....	7
9	Hledám s ID a testuji existenci.....	8
9.1.1	Zabezpečení na existence přímo ve twigu.....	8
10	Příklad Update.....	9
11	Hledáme podle id Použití DQL.....	9
11.1	Hledáme podle kraje.....	11
12	Mazání.....	11
13	Úprava entity – jak do entity Inzerat vložit sloupec -.....	12
14	Nova tabulka a propojeni, nastavení asociace.....	13
14.1	Výpis dat v asociaci.....	17
15	Použití SQL.....	19
16	Použití QueryBulder.....	20
17	DQL.....	20
18	Úprava Crud.....	20
19	Upozornění mazání.....	21
20	Rozdělení řádků formuláře a české pojmenování. Nezapomenout v use TextType.....	21
21	Vzhled formuláře.....	22

1 Database ručně

Verze Symfony 6.4

Vytvoříme projekt

composer create-project symfony/skeleton:"6.4.*" my_project_directory

Následně se přesuňte do projektu přes příkazy:

cd my_project_directory

composer require webapp

Tento balíček zahrnuje několik základních komponent a nástrojů, které vám umožní rychlý start s webovou aplikací v Symfony. Když tento příkaz spustíte, Composer stáhne a nainstaluje balíček spolu se všemi závislostmi a aktualizuje soubor `composer.json` o tento nový balíček.

Když použijete příkaz `composer require webapp` v projektu Symfony, nainstalují se následující komponenty:

`symfony/asset`**: Správa a optimalizace statických zdrojů (jako jsou obrázky, CSS a JavaScript).

`symfony/console`**: Umožňuje vytváření konzolových příkazů.

`symfony/flex`**: Vylepšuje správu balíčků pro Symfony projekty.

`symfony/form`**: Umožňuje snadnou tvorbu a zpracování formulářů.

`symfony/framework-bundle`**: Jádru aplikace Symfony.

`symfony/http-client`**: Asynchronní HTTP klient.

`symfony/intl`**: Podpora pro mezinárodní formáty (jako jsou měny, časová pásma, atd.).

`symfony/mailer`**: Odesílání e-mailů.

`symfony/messenger`**: Implementace zpráv a front.

`symfony/notifier`**: Posílání upozornění (např. e-mail, SMS).

`symfony/process`**: Spouštění systémových procesů.

`symfony/security-bundle`**: Bezpečnostní komponenty.

`symfony/serializer`**: Serializace a deserializace dat.

`symfony/string`**: Manipulace s textem.

`symfony/twig-bundle`**: Twig šablonovací engine.

`symfony/validator`**: Validace dat.

symfony/web-link**: HTTP/2 Server Push a pre-loading.

symfony/web-profiler-bundle**: Nástroje pro ladění webových aplikací.

symfony/yaml**: Práce se soubory YAML.

Tyto komponenty vám umožní rychle začít s plně funkční webovou aplikací. Je něco konkrétního, na co byste se chtěl zaměřit nebo potřebujete nějakou další pomoc?

2 Nastavení databáze:

- Otevřeme soubor .env
- DATABASE_URL podle vašeho databázového serveru
- Příklad použití: SQL je **MySQL** a vaše databáze se jmenuje **symfony_db**
DATABASE_URL="mysql://username:password@localhost:3306/symfony_db"
- Změňte username, password a symfony_db podle vašich údajů

Pozn pro xampp například

```
DATABASE_URL="mysql://pokus:pokus@127.0.0.1:3306/pokus?  
serverVersion=mariadb-10.4.28"
```

3 Vytvoření entity

Vytvoříme tabulku inzerat s následujícími sloupci:

id (vytvoří se automaticky), není nutné ji zadávat

pozice: varchar 255

plat: int

kraj: varchar 255

popisek: varchar 255 nebo typ clob (4 GB znaků)

pomocí příkazu

symfony console make:entity Inzerat

nebo

php bin/console make:entity Inzerat

Do MySQL pomocí phpmyadmin přidáme několik řádků.

4 Vytvoříme Controller

symfony console make:controller InzeratController

5 Managery pro práci s daty

5.1 EntityManager:

- **Účel:** EntityManager je hlavní třídou pro práci s Doctrine ORM. Používá se k provádění operací CRUD (Create, Read, Update, Delete) na entitách.
- **Použití:** Přímo spravuje entity a jejich stav. Můžete jej použít k vyhledávání, ukládání, aktualizaci a mazání entit.

Pro práci s jednou databází bych doporučil použití **EntityManageru**. Je to přímější a jednodušší způsob, jak spravovat entity a provádět operace CRUD. EntityManager poskytuje všechny potřebné funkce pro práci s databází bez nutnosti další abstrakce.

ManagerRegistry:

- **Účel:** ManagerRegistry je služba, která poskytuje přístup k více EntityManagerům a databázovým připojením. Je užitečná, pokud vaše aplikace pracuje s více databázemi nebo potřebuje různé EntityManagery pro různé účely.
- **Použití:** Umožňuje získat konkrétní EntityManager podle potřeby. Můžete jej použít k získání výchozího nebo specifického EntityManageru.

6 Zápis dat do databaze

Pro zápis přidat use do controlleru:

use App\Entity\Inzerat;

```
namespace App\Controller;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Attribute\Route;
use Doctrine\Persistence\ManagerRegistry;
use App\Entity\Inzerat;
```

7 Zapis do databaze

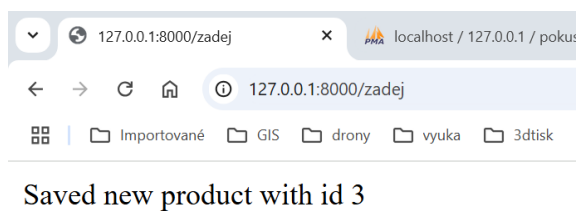
```
#[Route('/zadej2', name: 'zadej2')]
public function zadej2(EntityManagerInterface $entityManager): Response
{
    $inzerat = new Inzerat();
    $inzerat->setPozice('reditel"2');
```

```

$inzerat->setKraj('Plzensky2');
$inzerat->setPlat(147147);
$inzerat->setPopisek('Pokusne zadání');
// říct Doctrine, že chceme (případně) uložit tento objekt (ale neuloží
se hned)
$entityManager->persist($inzerat);
$entityManager->flush();
return new Response('Saved new product with id '.$inzerat->getId());
//return $this->render('inzerat/index.html.twig', [
//    'controller_name' => 'InzeratController',
])
}

```

Vložená data jsou pak vidět v databázi.



8 Čtení z databáze

Udělám v controlleru akci, která mi vypíše data

8.1 Použití EntityManageru

Nezapomenout v controlleru

```
use Doctrine\ORM\EntityManagerInterface;
```

8.1.1 Výpis pomocí Response

```

#[Route('/cteni2', name: 'cteni2')]
public function cteni2(EntityManagerInterface $entityManager): Response
{
    $id=3;
    $inzeraty = $entityManager->getRepository(Inzerat::class)->find($id);
    return new Response("id:".$inzeraty->getId()." pozice:".$inzeraty-
    >getPozice());
}

```

8.1.2 Použití findall a výpis pomocí response

```

public function cteni2(EntityManagerInterface $entityManager): Response
{
    $inzeraty = $entityManager->getRepository(Inzerat::class)->findall();

    $inzeratyPole = [];
    $output = '<html><body><h1>Product List</h1><ul>';
    foreach ($inzeraty as $inzerat)
    {
        $output .= '<li>' . $inzerat->getId() . ' - ' . $inzerat-
        >getPozice() . '</li>';
    }
    $output .= '</ul></body></html>';
}

```

```
return new Response($output);
}
```

← → ↺ 🏠 127.0.0.1:8000/cteni2

🗑️ | 📁 Importované 📁 GIS 📁 drony 📁 vyuka

Product List

- 1 - vedoucí
- 2 - pracovník
- 3 - reditel

Použití JsonResponse pro výpis

```
#[Route('/cteni2', name: 'cteni2')]
public function cteni2(EntityManagerInterface $entityManager): Response
{
    $inzeraty = $entityManager->getRepository(Inzerat::class)->findAll();

    $inzeratyPole = [];
    foreach ($inzeraty as $inzerat)
    {
        $inzeratyPole[] = [ 'id' => $inzerat->getId(), 'pozice' =>
        $inzerat->getPozice(), ];
    }
    return new JsonResponse($inzeratyPole);
}
```

← → ↺ 🏠 127.0.0.1:8000/cteni2

🗑️ | 📁 Importované 📁 GIS 📁 drony 📁 vyuka 📁 3dtisk

Vaformátovat ☐

```
[{"id":1,"pozice":"vedouc\u00ed"}, {"id":2,"pozice":"pracovn\u00edk"}, {"id":3,"pozice":"reditel"}]
```

8.1.3 Výpis pomocí twigu

```
#[Route('/cteni2', name: 'cteni2')]
public function cteni2(EntityManagerInterface $entityManager): Response
{
    $inzeraty = $entityManager->getRepository(Inzerat::class)->findAll();
    return $this->render('inzerat/cteni.html.twig', ['inzeraty' =>
    $inzeraty]);
}
```

```
#[Route('/cteni', name: 'cteni')]
public function cteni(ManagerRegistry $doctrine): Response
{
    $id=3;
    $entityManager = $doctrine->getManager();
    $inzeraty = $entityManager->getRepository(Inzerat::class)->findAll();
    return $this->render('inzerat/cteni.html.twig', ['inzeraty' =>
    $inzeraty]);
}
```

Twig

```
<h1>Výpis inzeratu</h1>
{{ dump(inzeraty) }}
{% for radek in inzeraty%}
    <p>{{ radek.pozice }}</p>
    <p> {{ radek.kraj}}</p>
{% endfor %}
```

8.1.4 Poznámka pro find

find(podminka pro primární klíč)

findby(podminka pro další sloupce)

```
$products = $repository->findBy(
    ['name' => 'Keyboard'],
    ['price' => 'ASC']
);
```

findall

8.1.5 Použití find()

```
#[Route('/cteni2', name: 'cteni2')]
public function cteni2(EntityManagerInterface $entityManager): Response
{
    $id=3;
    $inzeraty = $entityManager->getRepository(Inzerat::class)->find($id);
    return $this->render('inzerat/cteni.html.twig', ['inzeraty' =>
    $inzeraty]);
}
```

Pak twig vypadá takto

```
<h1>Výpis inzeratu</h1>
{{ dump(inzeraty) }}

    <p>{{ inzeraty.id }}</p>
    <p>{{ inzeraty.pozice }}</p>
    <p> {{ inzeraty.kraj}}</p>
```

8.2 Použití ManagerRegistry

```
use Doctrine\ORM\EntityManagerInterface;
```

Výpis inzeratu

```
App\Entity\Inzerat {#715 ▾  
  -id: 3  
  -pozice: "reditel"  
  -plat: 147147.0  
  -kraj: "Plzensky"  
  -popisek: "Pokusne zadani"  
}
```

3

reditel

Plzensky

9 Hledám s ID a testuji existenci

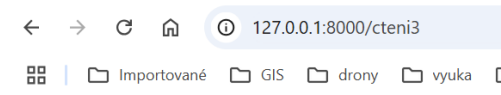
```
#[Route('/cteni3', name: 'cteni3')]  
public function cteni3(ManagerRegistry $doctrine): Response  
{  
    $id=10;  
    $entityManager = $doctrine->getManager();  
    $inzeraty = $entityManager->getRepository(Inzerat::class)->find($id);  
    if (!$inzeraty)  
    {  
        return new Response('zaznam neexistuje');  
    }  
    return $this->render('inzerat/cteni.html.twig', ['inzeraty' =>  
$inzeraty]);  
}
```

zaznam neexistuje

9.1.1 Zabezpečení na existence přímo ve twigu

```
#[Route('/cteni3', name: 'cteni3')]  
public function cteni3(ManagerRegistry $doctrine): Response  
{  
    $id=10;  
    $entityManager = $doctrine->getManager();  
    $inzeraty = $entityManager->getRepository(Inzerat::class)->find($id);  
  
    return $this->render('inzerat/cteni3.html.twig', ['inzeraty' =>  
$inzeraty]);  
}
```

```
<h1>Výpis inzeratu</h1>
{{ dump(inzeraty) }}
{% if inzeraty is empty %}
<p> Nebyly nalezeny žádné inzeraty
{% else %}
    <p>{{ inzeraty.id }}</p>
    <p>{{ inzeraty.pozice }}</p>
    <p> {{ inzeraty.kraj }}</p>
{% endif %}
```



Výpis inzeratu

null

Nebyly nalezeny žádné inzeraty

10 Příklad Update

```
#[Route('/update', name: 'update')]
public function update(EntityManagerInterface $entityManager): Response
{
    $id=1;
    $inzerat = $entityManager->find(Inzerat::class, 3);
    $inzerat->setPopisek('Budoucí ředitel');
    $entityManager->flush();
    return new Response('inzerat byl upraven');
}
```

Aktualizace objektu zahrnuje pouze tři kroky:

vyzvednutí předmětu z doktríny;

úprava objektu;

zavolání flush()správce entity.

11 Hledáme podle id Použití DQL

DQL (Data Query Language) je podmnožina SQL (Structured Query Language), která se zaměřuje na dotazování a získávání dat z databáze. Hlavním příkazem DQL je SELECT, který se používá k výběru dat z jedné nebo více tabulek

DQL také zahrnuje různé operátory a funkce, které umožňují manipulaci s vybranými daty, jako jsou WHERE, JOIN, GROUP BY, ORDER BY a agregační funkce jako COUNT(), SUM(), AVG()

má DQL několik omezení:

1. Přímý přístup k databázovým funkcím:

- SQL umožňuje přímé použití databázových funkcí, jako jsou COUNT(), SUM(), AVG(), a další. DQL je omezenější a některé specifické databázové funkce nemusí být dostupné.

2. Manipulace s databázovou strukturou:

- SQL podporuje příkazy jako CREATE TABLE, ALTER TABLE, DROP TABLE, které umožňují manipulaci s databázovou strukturou. DQL je zaměřený pouze na dotazování a práci s daty, nikoli na změny struktury databáze.

3. Optimalizace dotazů:

- SQL umožňuje pokročilé optimalizace dotazů, jako je použití indexů, optimalizačních tipů a dalších technik. DQL se zaměřuje na práci s entitami a objekty, takže tyto optimalizační techniky nejsou přímo dostupné.

4. Transakční řízení:

- SQL poskytuje příkazy pro řízení transakcí, jako jsou BEGIN, COMMIT, ROLLBACK. I když Doctrine podporuje transakce, DQL samotný nemá přímé příkazy pro transakční řízení.

5. Pokročilé spojování tabulek:

- SQL umožňuje složité spojování tabulek (joins), včetně LEFT JOIN, RIGHT JOIN, FULL JOIN. DQL podporuje spojování entit, ale může být omezenější v pokročilých scénářích spojování.

```
#[Route('/find-product-dql', name: 'find_product_dql')]
public function findProductDQL(EntityManagerInterface $entityManager):
Response
{
    $dql = 'SELECT p FROM App\Entity\Inzerat p WHERE p.id = :id';
    $query = $entityManager->createQuery($dql);
    $query->setParameter('id', 3);

    $product = $query->getSingleResult();

    if (!$product) {
        return new Response('Product not found.');
```

Nalezena pozice: reditel with price 147147

11.1 Hledáme podle kraje

Pokud při hledání pomocí DQL (Doctrine Query Language) najdete více řádků, můžete použít metodu `getResult` místo `getSingleResult`.

```
#[Route('/podlekraje', name: 'podlekraje')]
public function podlekraje(EntityManagerInterface $entityManager): Response
{
    $dql = 'SELECT i FROM App\Entity\Inzerat i WHERE i.kraj = :kraj ORDER
BY i.id desc';
    $query = $entityManager->createQuery($dql);
    $query->setParameter('kraj', 'Vysočina');
    $inzeraty = $query->getResult();
    if (!$inzeraty) {
        return new Response('No inzerat found.');
```

Nalezene inzeraty:
Pozice: vedoucí s platem 200000
Pozice: reditel"2 s platem 147147

12 Mazání

Pokud někde získáme entitu, zavoláme `remove` a smažeme.

```
#[Route('/delete', name: 'delete')]
public function delete(EntityManagerInterface $entityManager): Response
{
    $inzerat = $entityManager->find(Inzerat::class, 3);
```

```
$entityManager->remove($inzerat);
$entityManager->flush();
return new Response('inzerat byl smazan!');
}
```

← → ↺ ↻ 127.0.0.1:8000/delete

Importované GIS drony vyuka 3dtisk

inzerat byl smazan!

Nalezení entity pro mazání pomocí DQL

```
#[Route('/delete-inzerat-dql', name: '/delete-inzerat-dql')]
public function deleteInzeratDQL(EntityManagerInterface $entityManager):
Response
{
    $dql = 'SELECT i FROM App\Entity\Inzerat i WHERE i.id = :id';
    $query = $entityManager->createQuery($dql);
    $query->setParameter('id', 1);

    $inzerat = $query->getSingleResult();

    if (!$inzerat) {
        return new Response('Inzerat nenalezen.');
```

13 Úprava entity – jak do entity Inzerat vložit sloupec -

Do entity Inzerat.php přidáme nový sloupec kategorie

```
#[ORM\Column(length: 255)]
private ?string $kategorie= null;
```

Pak aktualizujeme v příkazovém řádku.

php bin/console make:migration

php bin/console doctrine:migrations:migrate

V databázi se přidal další řádek.

```
Case mismatch between loaded and declared class names: GopasBundle\Entity\inzerat vs GopasBundle\Entity\Inzerat

doctrine:generate:entities [--path PATH] [--no-backup] [-h|--help] [-q|--quiet] [-v|vv|vvv|--verbose] [-V|--version] [--ansi] [--no-ansi] [-n|--no-interaction] [-e|--env ENV] [--no-debug] [--] <command> <name>

C:\Users\smrcka\gopas>php bin/console doctrine:generate:entities GopasBundle:Inzerat
Generating entity "GopasBundle\Entity\Inzerat"
> backing up Inzerat.php to Inzerat.php~
> generating GopasBundle\Entity\Inzerat

C:\Users\smrcka\gopas>
```

Přidat do Inzerat.php funkce prohlednut. Pak se ještě spustit, aby se to projevilo v db.

```
C:\Users\smrcka\gopas>php bin/console doctrine:generate:entities GopasBundle:Inzerat
Generating entity "GopasBundle\Entity\Inzerat"
> backing up Inzerat.php to Inzerat.php~
> generating GopasBundle\Entity\Inzerat

C:\Users\smrcka\gopas>php bin/console doctrine:schema:update --force
Updating database schema...
Database schema updated successfully! "1" query was executed

C:\Users\smrcka\gopas>
```

A projeví se to v databázi.

14 Nova tabulka a propojení, nastavení asociace

Vytvoříme novou tabulku Kategorie

symfony console make:entity Kategorie

Pak

php bin/console make:migration

php bin/console doctrine:migrations:migrate

	#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra
☐	1	id 	int(11)			No	None		AUTO_INCREMENT
☐	2	kategorie	varchar(255)	utf8_unicode_ci		No	None		

Upravíme entitu Inzerát

```
<?php

namespace App\Entity;

use App\Repository\InzeratRepository;
use Doctrine\ORM\Mapping as ORM;

#[ORM\Entity(repositoryClass: InzeratRepository::class)]
class Inzerat
{
    #[ORM\Id]
    #[ORM\GeneratedValue]
    #[ORM\Column]
    private ?int $id = null;

    #[ORM\Column(length: 255)]
    private ?string $pozice = null;

    #[ORM\Column]
    private ?float $plat = null;

    #[ORM\Column(length: 255)]
```

```

private ?string $kraj = null;

#[ORM\Column(length: 255)]
private ?string $popisek = null;

#[ORM\ManyToOne(targetEntity: Kategorie::class, inversedBy: 'inzeraty',
cascade: ['persist'])]
#[ORM\JoinColumn(name: 'kategorie_id', referencedColumnName: 'id')]
private $kategorie;

public function getId(): ?int
{
    return $this->id;
}

/**
 * Set category
 *
 * @param \App\Entity\Kategorie $kategorie
 *
 * @return Inzerat
 */
public function setKategorie(\App\Entity\kategorie $kategorie = null)
{
    $this->kategorie = $kategorie;
    return $this;
}

/**
 * Get category
 *
 * @return \App\Entity\Kategorie
 */
public function getkategorie()
{
    return $this->kategorie;
}

public function getPozice(): ?string
{
    return $this->pozice;
}

public function setPozice(string $pozice): static
{
    $this->pozice = $pozice;

    return $this;
}

public function getPlat(): ?float
{
    return $this->plat;
}

public function setPlat(float $plat): static
{
    $this->plat = $plat;
}

```

```

        return $this;
    }

    public function getKraj(): ?string
    {
        return $this->kraj;
    }

    public function setKraj(string $kraj): static
    {
        $this->kraj = $kraj;

        return $this;
    }

    public function getPopisek(): ?string
    {
        return $this->popisek;
    }

    public function setPopisek(string $popisek): static
    {
        $this->popisek = $popisek;

        return $this;
    }
}

```

Pak upravíme tabulku Kategorie

```

<?php

namespace App\Entity;

use App\Repository\KategorieRepository;
use Doctrine\Common\Collections\ArrayCollection;
use Doctrine\Common\Collections\Collection;
use Doctrine\ORM\Mapping as ORM;

#[ORM\Entity(repositoryClass: KategorieRepository::class)]
class Kategorie
{
    #[ORM\Id]
    #[ORM\GeneratedValue]
    #[ORM\Column]
    private ?int $id = null;

    #[ORM\Column(length: 255)]
    private ?string $kategorie = null;

    #[ORM\OneToMany(targetEntity: Inzerat::class, mappedBy: 'kategorie')]
    private Collection $inzeraty;

    public function __construct()
    {
        $this->inzeraty = new ArrayCollection();
    }

    public function getId(): ?int

```

```

{
    return $this->id;
}

public function getKategorie(): ?string
{
    return $this->kategorie;
}

public function setKategorie(string $kategorie): static
{
    $this->kategorie = $kategorie;

    return $this;
}

public function getInzeraty(): Collection
{
    return $this->inzeraty;
}
}

```

Opět udělat migraci, pro aktualizaci v databázi

php bin/console make:migration

php bin/console doctrine:migrations:migrate

Pak doplníme v kontrolleru řádek pro přidání dat

```

#[Route('/zadej3', name: 'zadej3')]
public function zadej3(EntityManagerInterface $entityManager): Response
{
    $kategorie = new Kategorie();
    $inzerat = new Inzerat();
    $inzerat->setPozice('reditel 3');
    $inzerat->setKraj('Plzensky 3');
    $inzerat->setPlat(151515);
    $inzerat->setPopisek('Pokusne zadání');
}

```

```

    $kategorie->setKategorie("MHD");
    $inzerat->setKategorie($kategorie);

    // říct Doctrine, že chceme (případně) uložit tento objekt (ale neuloží
se hned)
    $entityManager->persist($inzerat);
    $entityManager->flush();
    return new Response('Saved new product with id '.$inzerat->getId());
    //return $this->render('inzerat/index.html.twig', [
    //    'controller_name' => 'InzeratController',

    dump($inzerat);
}

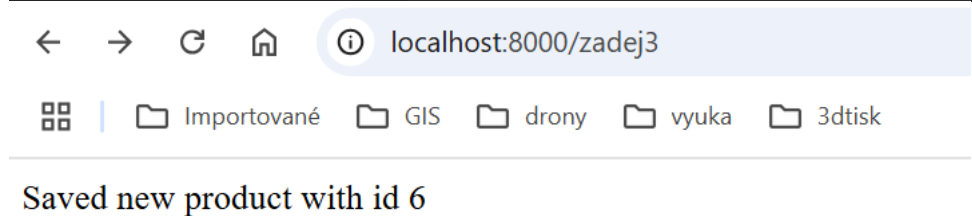
```

nahoře v controlleru musí být

```

use App\Entity\Kategorie;

```



The screenshot shows a web browser window with the address bar displaying 'localhost:8000/zadej3'. Below the address bar, there is a navigation bar with icons for 'Importované', 'GIS', 'drony', 'vyuka', and '3dtisk'. The main content area of the browser displays the text 'Saved new product with id 6'.

14.1 Výpis dat v asociaci

Controller

```

namespace App\Controller;

use App\Repository\InzeratRepository;
use App\Entity\Inzerat;
use Doctrine\ORM\EntityManagerInterface;
use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\JsonResponse;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Attribute\Route;
use Doctrine\Persistence\ManagerRegistry;
use App\Entity\Kategorie;

#[Route('/cteni3', name: 'cteni3')]
public function cteni3(ManagerRegistry $doctrine): Response
{
    $id=10;
    $entityManager = $doctrine->getManager();
    $inzeraty = $entityManager->getRepository(Inzerat::class)->find($id);

    return $this->render('inzerat/cteni3.html.twig', ['inzeraty' =>
    $inzeraty]);
}

```

pak twig

```
{# templates/inzerat/vypis3.html.twig #}

{% extends 'base.html.twig' %}

{% block title %}Inzeráty{% endblock %}

{% block body %}
    <h1>Inzeráty</h1>

    <table>
        <thead>
            <tr>
                <th>ID</th>
                <th>Pozice</th>
                <th>Plat</th>
                <th>Kraj</th>
                <th>Popisek</th>
                <th>Kategorie</th>
            </tr>
        </thead>
        <tbody>
            {% for inzerat in inzeraty %}
                <tr>
                    <td>{{ inzerat.id }}</td>
                    <td>{{ inzerat.pozice }}</td>
                    <td>{{ inzerat.plat }}</td>
                    <td>{{ inzerat.kraj }}</td>
                    <td>{{ inzerat.popisek }}</td>
                    {% if inzerat.kategorie is not null %}
                        {{ inzerat.kategorie.kategorie }}
                    {% endif %}
                </tr>
            {% endfor %}
        </tbody>
    </table>
{% endblock %}
```

← → ↻ 🏠 ⓘ localhost:8000/vypis3

🗑️ | 📁 Importované 📁 GIS 📁 drony 📁 vyuka 📁 3dtisk

Inzeráty

MHD

ID	Pozice	Plat	Kraj	Popisek	Kategorie
2	pracovník	50000	Praha	Pokusný 2	
4	reditel 2	147147	Vysočina	Pokusne zadání	
5	reditel 3	151515	Plzensky 3	Pokusne zadání	

15 Použití SQL

<https://symfony.com/doc/current/doctrine.html>

<https://www.doctrine-project.org/projects/doctrine-orm/en/current/reference/native-sql.html>

```
// src/Controller/InzeratController.php
namespace App\Controller;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Response;
use Doctrine\DBAL\Connection;
use Symfony\Component\Routing\Annotation\Route;

class InzeratController extends AbstractController
{
    private $connection;

    public function __construct(Connection $connection)
    {
        $this->connection = $connection;
    }

    /**
     * @Route("/inzeraty", name="inzerat_list")
     */
    public function index(): Response
    {
        // SQL dotaz pro získání dat z tabulky Inzerat
        $sql = 'SELECT plat, kraj, popisek, kategorie FROM Inzerat';
        $stmt = $this->connection->prepare($sql);
        $stmt->execute();
        $inzeraty = $stmt->fetchAll();

        // Předání dat do Twig šablony
        return $this->render('inzerat/vypis.html.twig', [
            'inzeraty' => $inzeraty,
        ]);
    }
}
```

15.1.1 Twig Šablona

```
{# templates/inzerat/vypis.html.twig #}
<table>
    <thead>
        <tr>
            <th>Plat</th>
            <th>Kraj</th>
            <th>Popisek</th>
            <th>Kategorie</th>
        </tr>
    </thead>
    <tbody>
```

```

        {% for inzerat in inzeraty %}
        <tr>
            <td>{{ inzerat.plat }}</td>
            <td>{{ inzerat.kraj }}</td>
            <td>{{ inzerat.popisek }}</td>
            <td>{{ inzerat.kategorie }}</td>
        </tr>
        {% endfor %}
    </tbody>
</table>

```

16 Použití QueryBulder

Doctrine také umožňuje psát složitější dotazy pomocí Doctrine Query Language (DQL). DQL je podobné SQL s tím rozdílem, že byste si měli představit, že se dotazujete na jeden nebo více objektů třídy entit (např. Product) namísto dotazování na řádky v tabulce (např. product).

Při dotazování v Doctrine máte dvě hlavní možnosti: psaní čistě DQL dotazů nebo použití Doctrine's Query Builder.

<https://www.doctrine-project.org/projects/doctrine-orm/en/current/reference/query-builder.html>

Doctrine také poskytuje [Query Builder](#), objektově orientovaný způsob psaní dotazů. Toto se doporučuje používat, když jsou dotazy sestavovány dynamicky (tj. na základě podmínek PHP):

```

/**
 * @Route("/cteniquerybulder")
 */
public function cteniquerybulder(Request $request)
{
    //chci získat všechna data pomocí QueryBulder
    $em = $this->getDoctrine()->getManager();
    $query=$em->createQueryBuilder()->select ('i')
        -> from (Inzerat::class,'i');
    $data = $query->getQuery()->getResult();
    return $this->render('default/inzeraty.html.twig',
        array('inzeraty'=>$data,)) ;
}

```

17 DQL

DQL není SQL

S pomocí **SQL** se dotazujete na jména databázových tabulek a jejich atributů. V jazyku **DQL** se místo toho používají jména Doctrine 2 entit a jejich členských proměnných. Výsledkem **SQL** dotazu je vždy jedna tabulka s jednotlivými řádky a sloupci. Výsledkem **DQL** dotazu jsou (zpravidla) načtené instance entit.

S pomocí DQL můžeme provádět pouze dotazy typu `SELECT`, `UPDATE` a `DELETE`.

18 Úprava Crud

Poznámky

19 Upozornění mazání

```
<a href="{{ path('firma_delete', {'firma': firma.id}) }}" class="btn btn-danger"
onclick="return confirm('Opravdu si přejete smazat vybranou firmu?')" title="Odstranit firmu">
```

Případně

```
<input type="submit" value="Delete" onclick="return confirm('Opravdu si přejete smazat vybranou firmu?')" title="Odstranit firmu">
```

20 Rozdělení řádků formuláře a české pojmenování. Nezapomenout v use TextType

```
<?php
```

```
namespace AppBundle\Form;
```

```
use Symfony\Component\Form\AbstractType;
use Symfony\Component\Form\Extension\Core\Type\TextType;
use Symfony\Component\Form\FormBuilderInterface;
use Symfony\Component\OptionsResolver\OptionsResolver;
```

```
class StrojType extends AbstractType
{
```

```
    /**
     * {@inheritdoc}
     */
    public function buildForm(FormBuilderInterface $builder, array $options)
    {
        $builder->add('nazev', TextType::class, [
            'label' => 'Název',
            'required' => true,
        ]);
    }
}
```

```

        $builder->add('cena',TextType::class, [
            'label' => 'Cena',
            'required' => true,
        ]);

    }/**
     * {@inheritdoc}
     */
    public function configureOptions(OptionsResolver $resolver)
    {
        $resolver->setDefaults(array(
            'data_class' => 'AppBundle\Entity\Stroj'
        ));
    }

    /**
     * {@inheritdoc}
     */
    public function getBlockPrefix()
    {
        return 'appbundle_stroj';
    }

}

```

21 Vzhled formuláře

```

{% extends 'base.html.twig' %}
{% block stylesheets %}
<link rel="stylesheet"
href="{{ asset
('https://maxcdn.bootstrapcdn.com/bootstrap/4.5.2/css/bootstrap.min.css') }}"
">
    {# {% form_theme form 'bootstrap_4_layout.html.twig' %} #}

    {% form_theme form 'bootstrap_4_horizontal_layout.html.twig' %}
{% endblock %}
{% block body %}
<div class="container">
    <h1>Stroj creation</h1>

    {{ form_start(form) }}
    <div style="width: 800px">
        {{ form_widget(form) }}
    </div>
    <input type="submit" value="Create" />
    {{ form_end(form) }}

    <ul>
        <li>
            <a href="{{ path('stroj_index') }}">Back to the list</a>
        </li>
    </ul>

```

```
</ul>
</div>
{% endblock %}
```

Stroj creation

Název

Cena

Create

- [Back to the list](#)